

An Exploration of Machine Learning for zero-day detection in network Intrusion Detection Systems

Omar Hyder

August 2026

Abstract

Machine-learning (ML) intrusion detection systems (IDS) routinely report $> 99\%$ accuracy, but on random splits that expose every attack type during training. Held-out splits with unseen attack classes in the test set cause dramatic declines in model performance. With exploit deployment now routinely preceding patches and AI compressing attacker timelines[5], zero-day detection capability is the true deployment-relevant metric, not known-attack accuracy. This write-up seeks to investigate IDS performance upon introduction to unseen, zero-day attacks. Two experiments were conducted on the CIC-IDS2017 dataset: a random split and an unseen-class split, comparing XGBoost, Isolation Forest (iForest) and Local Outlier Factor (LOF) models. With all attack classes seen, XGBoost reaches a false-positive rate (FPR) $\sim 1\text{-}2\%$ and a macro attack recall of 0.86 (weighted 0.99). With none seen, macro attack recall falls to 0.23 (weighted 0.01). Unsupervised anomaly detectors recover 0.67-0.76 macro attack recall at the cost of 37-46% FPR. The relative ranking of iForest and LOF anomaly detectors reverses between validation and test depending on which unseen attacks are present; even zero-day focused evaluation can mislead. The gap between known-attack and zero-day performance is large, and appears to persist across architectures. The results argue for held-out class splits and greater emphasis on zero-day capabilities to be standard in future ML IDS research.¹

¹Notebooks: <https://github.com/OmarHyder07/ML-IDS/tree/main>

Contents

1	Introduction	3
1.1	Current State of ML-IDS Research	3
1.2	CIC-IDS2017	4
2	Supervised performance under a random split	5
2.1	Shortcut learning?	6
2.2	Imbalance Experiment	7
2.3	Verdict	10
3	Unsupervised performance on an unseen-class split	11
3.1	Day-by-day evaluation	11
3.2	Anomaly Detection	12
3.3	The evaluation paradox	14
3.4	Verdict	16
4	Conclusion	17
4.1	Limitations	17
4.2	Future Work	18
A	Networking Glossary	19
A.1	Core Concepts	19
A.2	CIC-IDS2017 features	19
A.3	Notes on Attack Classes	21
B	Machine Learning Glossary	22
B.1	Core Concepts	22
B.2	Evaluation Metrics	22
C	CIC-IDS2017 Construction-Issues	23

1 Introduction

Intrusion Detection Systems (IDS) are integral to network security, acting as a first layer of defense by flagging malicious traffic. Threat actors are increasingly leveraging AI for productivity, especially during early attack phases like reconnaissance[5].

Yet, developments in machine learning (ML) also lead to developments in IDS technology beyond traditional methods such as firewalls and access control mechanisms[4].

This report does not require previous ML or networking experience for understanding. Appendices are provided with glossaries, notes on attack types and ML evaluation metrics.

1.1 Current State of ML-IDS Research

Many highly optimized systems, such as the ensemble-based IDS proposed by Patil et al. (2025)[7], achieve near-perfect metrics (e.g., 99.98% accuracy) on the Canadian Institute for Cybersecurity (CIC) dataset CIC-IDS2017. They use a randomly stratified 70/30 split. A random split of data causes individual attack types to be present in both training and test sets, i.e. the model is not tested on any unseen attacks. Moczkoan and Ragab (2026)[6] propose the use of temporal architectures such as transformers. They too used CIC-IDS2017 on a random split. The aforementioned dataset is split into 8 comma separated values (CSV) files across 5 simulated days. Each day contains "flow" data with unique attack types, for example, only Thursday contains web attacks such as structured query language (SQL) injection and cross-site scripting, while only Friday contains distributed denial-of-service (DDoS) flows. The key issue being, as they test on the exact same days and attack types they trained on, these metrics do not reflect chronological generalisability or vulnerability to zero-day events. In fact, in the realistic evaluation section of Moczkoan and Ragab's paper, they evaluate their model and 8 other deep learning (DL) and ML architectures under a time-based split (unseen attacks in test set), finding macro-F1 collapsed to ~ 0.50 for all models - not nearly suitable for a deployment-ready IDS. Exploits have remained the leading initial infection vector for the sixth year in a row, at 32% of investigations, and many of the vulnerabilities used for initial access were exploited as zero-days, weaponised before a patch existed[5]. While many papers in the field use highly sophisticated, frontier architectural designs, the use of a random split greatly detracts from some of the achieved near-perfect evaluation metrics.

The inadequacy of closed-set evaluation (training and test have identical label sets) is not unknown. Open-set (containing unknown classes in test) recognition approaches to intrusion detection date back to at least 2017 with work by Cruz et al.[2], and work from 2026 focusing on IDS for Internet of Things (IoT) confirms collapse under held-out-class splits on IoT datasets[1]. Despite this, high-visibility results on CIC-IDS2017, such as the work of Patil et al. from 2025[7] continue to use random splits. This report demonstrates that mod-

ern IDS research should reflect our current uncertain and volatile technological landscape by placing greater emphasis on zero-day detection. It contributes a controlled comparison of supervised and unsupervised methods under identical conditions on CIC-IDS2017, and documents a further hazard: model rankings themselves reverse depending on which unseen attacks appear in evaluation.

1.2 CIC-IDS2017

The CIC-IDS2017 dataset was generated using a network environment with various clients, using open-source tooling designed to simulate both benign and malicious traffic[8]. It is a modern, flow-based, realistic dataset. I previously worked on supervised methods on the older NSL-KDD dataset, which is much simpler, providing less realistic model evaluation. There are ~ 2.8 M flows from 8 CSV files, spread across a simulated working week, Mon-Fri, with ~ 80 features and 15 distinct classes (flow data extracted from packet captures).

Pre-processing Pre-processing is handled in the following steps:

1. All 8 CSVs are joined into one set.
2. If using a random split: rows are de-duplicated to avoid leakage (duplicate examples appearing in training and test sets).
3. If using a time-based split: rows are not de-duplicated - duplicate examples in train and test reflect real network traffic.
4. Column names are stripped of leading whitespace and non-UTF8 characters are normalised.
5. **inf** values are replaced with **NaN**.
6. Rows with **NaN** values are dropped (2,867 rows). Notably, this does not remove examples from any ultra-rare classes.
7. Columns are downcast from *float64* to *float32* where relative error $< 1e-07$.

Class	n
Benign	2,095,051
DoS Hulk	172,846
DDoS	128,014
PortScan	90,694
DoS GoldenEye	10,286
FTP-Patator	5,931
DoS slowloris	5,385
DoS Slowhttptest	5,228
SSH-Patator	3,219
Bot	1,948
Web Attack Brute Force	1,470
Web Attack XSS	652
Infiltration	36
Web Attack SQL Injection	21
Heartbleed	11

Table 1: Number of examples of each class in the entire dataset post de-duplication.

2 Supervised performance under a random split

Reproduction of high evaluation metrics on a random split of data using the supervised ensemble eXtreme gradient boosting (XGBoost) method.

The CIC-IDS2017 dataset contains extreme class-imbalance (see table 1). Models trained on such imbalanced datasets will be unable to generalise to minority classes, while biasing dominant ones. The accuracy metric is misleading here; a model could perform extremely well on only the top 4 classes listed while collapsing on the rest, and due to the extremity of the imbalance still achieve high accuracy. Per-class recall, FPR and other metrics will be displayed instead. Techniques for handling imbalance are imperative for reliable performance on small classes, thus the following experiment is conducted to compare various strategies.

Features and Leakage Permutation importance measures how test performance drops when a feature’s values are shuffled. This is a trustworthy signal as it captures non-linear relationships between features and classes. It can *under-report* a feature when a collinear partner covers for it, as XGBoost will split on one and leave the other apparently unimportant simply based on which it got to first. This is a limitation. Low-ranked features that are strongly correlated with high-ranked features won’t be listed. To account for this limitation, features were analysed in groups rather than as individual features. The following permutation importance results are of a ”baseline” XGBoost model trained with no imbalance handling:

- #1 Destination Port
- Connection setup fingerprints - #2 Init_Win.bytes_forward and #5 backwards. Attack tools often use non-standard stacks that differentiate them from normal traffic.
- Traffic volume and shape - #3 Total Length of Fwd Packets, #4 Fwd Header length. Also highly rated: Packet Length Mean, Fwd/Bwd Packet Length Max/Std, Average Packet Size, Bwd Header Length. These show how much data flowed and how it was distributed. Floods, scans and brute-force attacks each have characteristic volume signatures which the model picked up on.
- Timing - #6 Fwd IAT Min, also Flow IAT Min, Fwd IAT Std, Flow Packets/s. Gaps between packets and packet rate. Tool-generated attacks have unnaturally regular or extreme timing.

2.1 Shortcut learning?

CICIDS-2017 was generated in a controlled lab with attacks run against a specific service on their standard port. FTP-Patator against 21, SSH-Patator against 22. Web attacks against 80. Thus, the destination port feature acts almost as a direct label for some attack classes. A model can "detect" FTP-Patator by just learning the mapping of port 21 to the attack type. This is known as *shortcut learning*, and prevents the model from learning the true pattern of the attacks, causing issues in actual deployment. Attacks can target non-standard ports so a model trained with the destination port feature will not be reliable in deployment. Engelen et al. discuss the methodology used to create synthetic traffic in the dataset: "... in all cases these features do not have any semantic connection with the intended malicious activity, but rather stem from the specifics of the used attack tools." [3]

The offending features they list are: Destination Port, Init_Win.bytes_forward, Fwd Header Len, Total Fwd Pkt, Min Seg Size Fwd, and the flow-separators SYN Flag Count and Bwd Pkt Len Mean. This is corroborated by my above permutation importance analysis. The most important feature groups could all be evaluated as quirks of an attack generating tool.

True baseline To prevent unrealistically high evaluation metrics, all future models will be trained on a subset of the feature set excluding the aforementioned shortcut-learning features.

Results in table 2 (next page). The effect concentrated almost entirely on the Bot class (n=403), whose recall fell from 0.78 to 0.45; possibly the only class materially dependent on the shortcut features. SSH-Patator (n=639) fell marginally, 1.00 to 0.95. Higher-frequency classes were unaffected, suggesting that sufficient data lets the model learn genuine attack patterns instead of tool fingerprints.

Freq. band	Classes	n (test)	Recall (all feat.)	Recall (no shortcuts)	Δ
—	Benign	418,794	~ 1.00	~ 1.00	≈ 0.00
>10,000	DDoS, PortScan, DoS Hulk	78,599	~ 1.00	~ 1.00	≈ 0.00
>1,000	DoS GoldenEye/Slowhttptest/slowloris, FTP-Patator	5,298	~ 1.00	~ 0.99	-0.01
>100	Web Brute Force, Web XSS, Bot, SSH-Patator	1,452	~ 0.72	~ 0.63	-0.09
>0	Heartbleed, Infiltration, Web SQL Injection	15	~ 0.60	~ 0.53	-0.07

Table 2: Per-frequency-band recall with and without the shortcut-learning features (Destination Port, Init_Win_bytes_fwd, Fwd Header Len, Total Fwd Pkt, Min Seg Size Fwd, SYN Flag Count, Bwd Pkt Len Mean).

Generally, recall steeply declines with low-frequency classes. Further hyper-parameter tuning would be fruitless here as declining recall is indicative of an *imbalance* issue, not model failure.

2.2 Imbalance Experiment

Setup A controlled comparison of imbalance handling techniques. The following properties are constant across each experiment (including the previous baseline models):

- Model choice (XGBoost).
- Hyper-parameters (300 estimators, 0.1 learning rate, 20 early-stopping rounds, 'hist' tree method, fixed random state).
- Identical training and test sets.
- Resampling fit on training data only.
- Train/validation/test split of 60/20/20.

Metrics The meaningful metrics are per-attack-class recall, but since there are many attack classes, 4 macro metrics will be reported with individual class performance discussed where appropriate:

- **FPR** Proportion of benign flows flagged as attacks.
- **Alert Precision** Proportion of alerts/flags that are actually attacks.
- **Macro Attack Recall** unweighted mean of all classes recall, excluding benign.
- **Weighted Attack Recall** Proportion of attack flows that are caught.

Type	Technique	FPR	Alert precision	Macro attack recall	Weighted attack recall
Baseline	—	~0.00	0.99	0.82	0.99
Weights	Sample weights	0.01	0.93	0.86	~1.00
Under	Cluster-centroid	0.05	0.81	0.86	~1.00
Under	Random → ENN	0.04	0.82	0.81	0.99
Over	Naive random oversampling	~0.00	0.99	0.82	0.99
Over	SMOTE	~0.00	0.99	0.83	0.99
U-O-U	Random → SMOTE → ENN	0.02	0.91	0.86	0.99

Table 3: Imbalance techniques: Baseline (none) vs. Sample Weights vs. Under-Sampling vs. Over-Sampling vs. Over-Under synthesis, with XGBoost, common hyper-parameters, same train and test set, resampling fit on train only.

FPR is a very insightful first metric. A real security operations center (SOC) team would be overwhelmed at anything above ~0.05 FPR; constant false-alarms would cost a huge amount of time and money. Over-sampling techniques left FPR near baseline, under-sampling cost 0.04. Sample weights is thus a clear winner. This method does not over or under sample, rather it weighs lower-frequency classes as more important in training. It has the best trade off of high macro attack recall (0.86) without increasing the FPR too high (0.01). Despite this, further analysis will focus on the synthesis of U-O-U. Under-sampling methods drastically reduce training time, and the synthesis brought this with only a modest increase to FPR from the sample weights method. This is highly practical: if retraining with new, real-world network traffic, speed and minimal compute use will be highly important. Oversampling also has the most interesting failure mode for dissection.

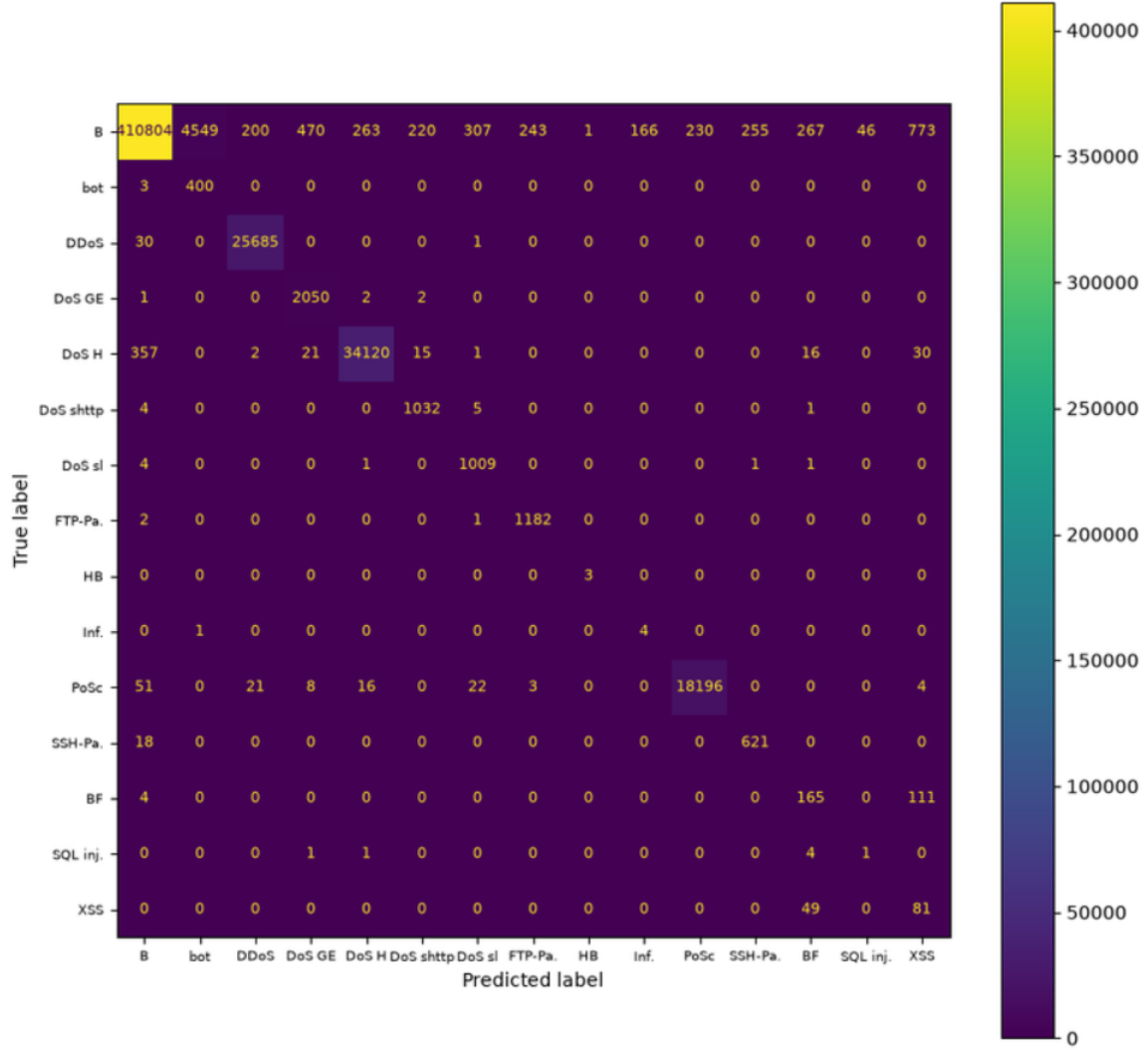


Figure 1: Confusion matrix of U-O-U synthesis model

From analysing the confusion matrix of the imbalance technique synthesis model (Figure 1), we see the majority of errors arise from false-positives, where benign flows are misclassified as anomalous. This is most striking with the "Bot" class, with 4,549 benign flows classified as Bot. This is unsurprising. A botnet tool creates seemingly benign pings to a central command server, which are unnaturally regular. This leads to a "flaw" with oversampling - the model will over-predict rare classes. Here, it will be more sensitive towards the botnet fingerprint, so much so that it assigns many benign flows to the Bot class. Thus, recall for the bot class is near perfect, while precision is 0.08. The baseline had

Freq. band	Classes	n (test)	Recall (baseline)	Recall (synthesis)	Δ
—	Benign	418,794	~ 1.00	~ 0.98	-0.02
$>10,000$	DDoS, PortScan, DoS Hulk	78,599	~ 1.00	~ 0.99	-0.01
$>1,000$	DoS GoldenEye/Slowhttptest/slowloris, FTP-Patator	5,298	~ 0.99	~ 0.99	≈ 0.00
>100	Web Brute Force, Web XSS, Bot, SSH-Patator	1,452	~ 0.63	~ 0.79	$+0.16$
>0	Heartbleed, Infiltration, Web SQL Injection	15	~ 0.53	~ 0.65	$+0.12$

Table 4: Per-frequency-band unweighted recall comparison of baseline and with synthesis of imbalance handling techniques applied.

recall of the bot class at 0.45. The synthesis brought perfect recall at the cost of 4,549 false alarms. However, since the FPR is generally low, perhaps a security team could handle the precision-recall tradeoff. Missing an attack could have catastrophic consequences, however the SOC team would have to decide if they have enough resources to deal with thousands of false-positives each day.

SQL Injection is particularly bad, with almost every example misclassified. This is also unsurprising. Web attacks occur from malicious data within the *payload* of packets. Since CIC-IDS2017 contains statistics about flows (such as payload size), without any of the actual data within the payload, the SQL injection and cross-site scripting (XSS) attacks will be difficult to differentiate from brute force attacks - all web-attacks exploiting form input. How can we tell if data entered to a form is a password attempt, SQL queries or a malicious script if we have no access to the data itself? Furthermore, are imbalance handling techniques sufficient to overcome such extreme scarcity? With SQL Injection having only 9 examples in training, there is a lack of information required for oversampling methods to infer any unique examples from - perhaps imbalance handling is only capable of aiding a class' recall where sufficient examples exist.

On the other hand, the only low-frequency class still near zero recall is SQL Injection, which is payload-invisible, and the two mid-frequency web classes (BF $n=280$, XSS $n=130$) sit at ~ 0.6 while similarly sized non-web classes are near 1.0. Heartbleed (3 test examples) got 3/3, Infiltration 4/5.

Thus we can say SQL injection is confounded between both scarcity and payload-invisibility, while Heartbleed and Infiltration - scarce but visible - show scarcity alone isn't fatal. Feature visibility is as dangerous as extreme imbalance.

2.3 Verdict

While imbalance handling techniques created marked improvements, reaching near perfect weighted attack recall without raising the FPR too high, the aforementioned results show nothing of the model's capacity to classify unseen attacks. Thus, a relevant direction of investigation would be to evaluate the same model on a split in which the test set contains unseen attack types.

3 Unsupervised performance on an unseen-class split

The following section demonstrates the collapse of performance when unseen attacks are present in the test set, and explores the performance of various unsupervised algorithms applied to a day-by-day split of data.

3.1 Day-by-day evaluation

The previous split; joining all data together into one large set, then randomly splitting for train and test, allows the models to see every attack type in training. In real deployment, we won't be able to train the model on the attacks that appear in the future.

Setup Training on Monday-Wednesday, testing on Thursday-Friday. Of the 7 attack types present in the test set (Bot, DDoS, PortScan, Infiltration, all Web Attacks: XSS, Sql Injection, BruteForce), all are unseen in the training set. In line with the previous section, evaluation of an XGBoost model under this split will be conducted first. Same hyper-parameters, same removal of shortcut learning features. Previously the entire set was de-duplicated. Here, duplicate rows between the time-based split are fine, as this is a genuine rather than an artificial split. Thus, it will be skipped. As there are unseen categories, transformation from multiclass to binary classification - benign vs. anomalous - will be applied, without imbalance handling.

	Random-split, U-O-U		Unseen-split	
Class	n (test)	Recall	n (test)	Recall
Benign	418,794	0.98	870,343	0.99
Anomaly	85,364	0.86	291,011	0.23
Bot	403	0.99	1,956	~0.00
Infiltration	5	0.80	36	~0.00
PortScan	18,321	0.99	158,804	~0.00
DDoS	25,716	1.00	128,025	~0.00
SQL Injection	7	0.14	21	0.33
Brute Force	280	0.59	1,507	0.60
XSS	130	0.62	652	0.69

Table 5: Supervised model (XGBoost) performance comparison over random and unseen-class splits of data. Anomaly recall is macro (unweighted). Note, the number of examples in test differs between splits; the time-based unseen split contains *every* listed attack example, whereas the random split contains only those randomly selected for the test set after de-duplication. Some unseen frequencies differ from table 1 due to no de-duplication. For the unseen-split, recall was split by class post-inference, as inference was binary-classification.

Results Table 5 shows steep drops in anomaly recall. From macro-average attack recall of 0.86 on the random split to 0.23; with weighted attack recall, a drop from 0.99→0.01. Every class falls to ~ 0.00 recall, except the web attacks which paradoxically achieve slight improvements. This could be an artifact of resemblance to Wednesday’s DoS attacks, however, future zero-day attacks may not have any similarity to today’s known attacks. The results motivate an exploration of unsupervised methods for zero-day detection.

3.2 Anomaly Detection

Supervised models structurally can not recognise what they have never seen. An approach that models *normal* and flags deviation doesn’t depend on having seen attacks, thus it fits as the next step of investigation for ML as an IDS.

Two methods will be evaluated: Isolation Forest (iForest) and Local Outlier Factor (LOF), both trained on entirely benign flows. They have different mechanisms of anomaly detection, thus it will be of interest to compare the per-class performance of each.

- **iForest** finds points which can be separated with random cuts, i.e. globally sparse outliers and weird one-offs. For example, the PortScan class (in test), with its 1000s of near-identical flows, is likely to evade an iForest’s detection. Whereas, infiltration and Heartbleed (in CV) attacks are more likely to be flagged due to their global uniqueness.
- **LOF** uses a density-based method with a different inductive bias - it asks if examples in the test set are substantially less dense than the training set. Due to the presence of unseen categories in the test set, along with training on benign only, training is conducted with novelty. This allows the LOF model to point out *locally unique* examples, i.e. attacks which seem benign - ones which an iForest would miss. Hence it should excel on brute force (Patator) (in CV), PortScan and DDoS.

Method For both models, conduct one sweep of the *contamination* parameter, and choose a value based on a precision-recall tradeoff. Specifically, choosing for highest recall where precision is at least 0.40 (note: this was an arbitrary choice. If you are reproducing results, perhaps select for F1, or F2 for recall bias). Contamination tells the model the proportion of anomalous examples present in the test / deployment data. The sweep will be conducted on a cross-validation set taken from the training days, thus avoiding leakage. Attack proportions are unpredictable however, volume varies by day and different attack types may have shallower or heavier traffic. Yet, this is a very real, deployment-esque limitation that should not be avoided. All of the attacks in Mon-Wed are concentrated in the cross-validation set, so the contamination proportion found may be artificially high, at least relative to test.

- iForest is an ensemble method which requires minimal hyper-parameter tuning. It also requires, if not benefits, from no data scaling.

- LOF relies on "distances", and thus does require scaling (if one feature is measured in thousands, it will apply a much greater range of distances than a feature measured in tens). Robust Scaling will be applied to every numerical column, fit on the training set. Robust scaling uses median and inter quartile range to replace mean and standard deviation in z-scaling, making it more robust to extreme points. Due to LOF's quadratic time complexity, I will take a sub-sample of 200,000 benign examples from training. A single sweep of the number of neighbours parameter will be conducted.

Both were trained entirely on benign examples from training. All anomalous flows in training will be in the cross-validation set.

	FPR	Alert precision	Macro attack recall	Weighted attack recall
XGBoost	0.01	0.19	0.23	0.01
iForest	0.37	0.29	0.67	0.46
LOF	0.46	0.40	0.76	0.91

Table 6: XGBoost vs. iForest vs. LOF

	Bot	Infil.	PortScan	DDoS	Sql Inj.	Brute Force	XSS
n (test)	1,956	36	158,804	128,025	21	1,507	652
XGBoost	~0.00	~0.00	~0.00	~0.00	0.33	0.60	0.69
iForest	0.29	0.97	0.12	0.87	0.57	0.91	0.97
LOF	0.47	0.81	0.99	0.81	0.43	0.88	0.96

Table 7: Per-class recall; iForest vs. LOF; XGBoost values copied from table 5

Results Across the board jumps in anomaly recall for both iForest and LOF from XGBoost, with sharp FPR increases. A real SOC team would be overwhelmed at 0.05 FPR with this volume of traffic. My predictions (on test) with per-class performance based on their inductive biases were correct, except for DDoS. Although LOF was expected to perform well on DDoS due to its density-based approach, iForest achieved superior detection, suggesting that the DDoS samples were more readily distinguished as globally isolated regions of the feature space than as local density anomalies relative to benign traffic. Comparing the two models at equal FPRs, however, reveals a deeper problem, examined next.

3.3 The evaluation paradox

Cross validation and test each contain only unseen attacks, and so both are legitimate zero-day evaluations. However, they disagree on which model is better.

Both models' "score" functions (higher number represents higher confidence of being anomalous) were percentile rank transformed to scale each score in the range $[0, 1]$. Then, sweeping through thresholds, FPR vs. weighted attack recall was plotted for validation and test.

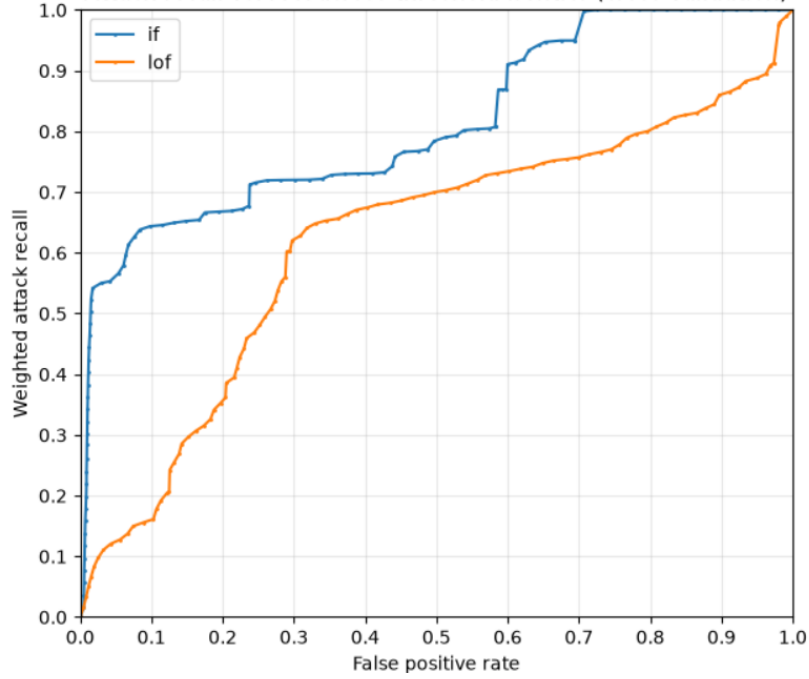


Figure 2: Weighted attack recall vs. FPR for iForest and LOF on cross validation. Unseen attacks: Brute Force with Patator, DoS attacks, Heartbleed.

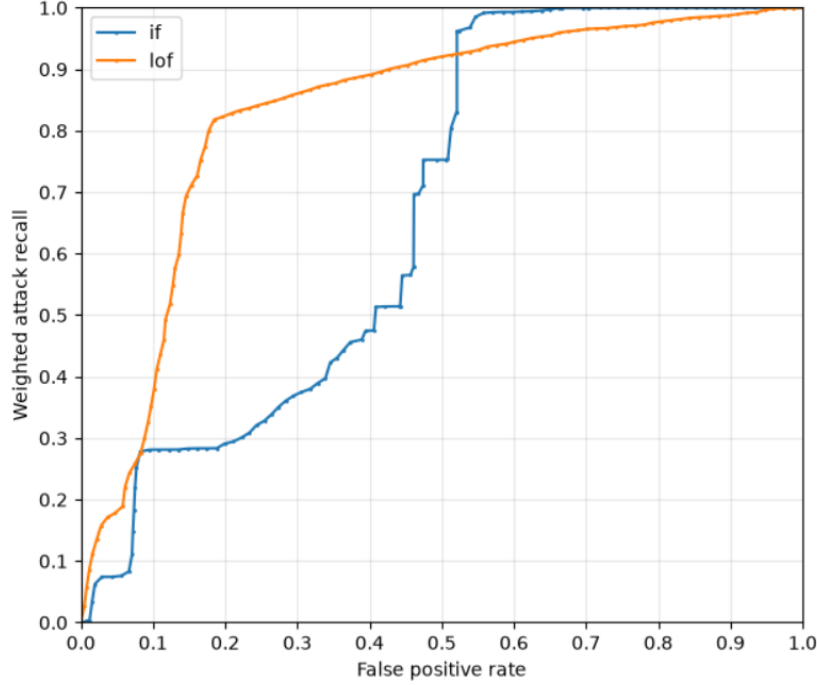


Figure 3: Weighted attack recall vs. FPR for iForest and LOF on test. Unseen attacks: Web attacks, Infiltration, DDoS, PortScan, Botnet.

FPR	iForest (CV)	LOF (CV)	iForest (test)	LOF (test)
0.05	0.56	0.13	0.08	0.18
0.10	0.64	0.16	0.28	0.38
0.20	0.67	0.36	0.29	0.83

Table 8: Weighted attack recall at fixed FPR operating points, read from figures 2 and 3

At every FPR up to ~ 0.52 the validation winner and the test winner disagree. Validation’s unseen attacks (Patator brute force, DoS attacks, Heartbleed) skew towards globally isolated flows, which suits iForest’s isolation bias. Test is 55% PortScan, whose thousands of near-identical flows form a dense region that evades random cuts, but stands out to a density method with novelty. Each benchmark rewards the inductive bias that happens to match the given unseen attacks.

This contrast exposes another difficulty of zero-day detection. By definition

they are unseen, unknown. Here, with unseen attacks in cross validation, iForest is clear as the superior model, and on test, with a new set of unseen attacks, the opposite is true. Single held-out splits are insufficient even when avoiding random-splits; aggregate metrics like macro recall can conceal that a model is only winning because of one dominant class. Every metric must be picked apart - individual class performance must be analysed. Specific types of attacks may be more easily detected by certain architectures. These links must be carefully scrutinised in order to prevent complacency. Unknown attacks may have certain fingerprints and quirks which allow them to evade detection by specific model architectures with different notions of an anomaly and different inductive biases, so in evaluation we must consider as broad a spectrum of attack classes as possible.

3.4 Verdict

While the unsupervised anomaly detection methods greatly improved recall on unseen attack classes, they achieved catastrophic precision and FPRs. This is an extreme trade-off, and neither the iForest or the LOF methods would be deployable in a real environment.

4 Conclusion

This project tested a variety of supervised and unsupervised methods as Intrusion Detection Systems. With regards to supervised methods, the crucial finding was the collapse when introduced to unseen attacks in the test set. With all attack classes seen in training, XGBoost reaches $\sim 85\%$ macro attack recall, at $\sim 1\text{-}2\%$ FPR; with none seen, macro attack recall falls to 23% and weighted to $\sim 1\%$. Unsupervised methods partially recover novel-attack detection with substantial FPR increases.

Furthermore, zero-day evaluation alone was shown to be unstable. Benchmarks containing only one composition of classes for anomaly detector comparison are unreliable. A key step in honest zero-day detection evaluation is not just to focus on holding out attack classes, but to analyse how performance changes when individual attacks are or are not held out.

Leveraging AI, attacker timelines are becoming increasingly compressed[5]. Human-speed SOC triage of alerts is under pressure, and thus high FPR IDS models are not deployment-ready. The difficulty with zero-day attacks persists across supervised and unsupervised methods. Readers should more deeply scrutinise metrics which measure only known-attack classification and ignore zero-day capability. The gap between the two is large and under-examined.

Model	Split	Test attack classes seen in training	FPR	Alert precision	Macro attack recall	Weighted attack recall
XGBoost (synthesis)	Random	14 of 14	0.0191	0.91	0.86	0.99
XGBoost	Unseen	0 of 7	0.0115	0.19	0.23	0.01
iForest	Unseen	0 of 7	0.3744	0.29	0.67	0.46
LOF	Unseen	0 of 7	0.4602	0.40	0.76	0.91 ¹

¹ High weighted attack recall of LOF is primarily a result of its success on the PortScan class, which is 55% of the test set.

Table 9: Comparison of supervised and unsupervised on Random and Unseen splits

4.1 Limitations

Construction issues with CIC-IDS2017 dataset. See appendix C.

Due to limited compute, a small subsample was taken for the LOF method. As LOF was only trained on benign flows, this subsample likely had a small but real impact on how well it could learn normal traffic. Furthermore, only one trial was conducted of iForest, which is a stochastic process dependent on random seeding. With more compute, a sweep of at least 3-5 random seeds would have been conducted, potentially exposing much more variance of results.

Greater care could have been placed on feature-scaling and hyper-parameter tuning. Small optimisations were seen as fundamentally incapable of breaching the ceilings of extreme imbalance, scarcity of examples and unseen classes.

However, with more time, comparisons would have been fairer in a situation where each model was fully tuned.

Only one dataset was used, thus whether or not results about models, supervised performance or unsupervised performance, imbalance handling techniques etc. can generalise any further is uncertain.

4.2 Future Work

In a future project or study, I could evaluate a wider range of model architectures, including reconstruction-based methods such as autoencoders. It would be valuable to see what impact choice in model architecture has in zero-day detection. However, as stated in 1.1, Moczko and Ragab showed the failure of many ML and DL architectures over a time-based split. They say: "architectural sophistication alone is a poor predictor of real-world IDS performance" [6].

I would test on multiple other datasets to see if my findings generalise further. I would seek other datasets not for differing feature sets, but for the variety of attack classes represented.

I could also use the leave-one-attack-family-out method. Here, this would entail the creation of 14 unique test sets where only one attack class is held-out, thus allowing me to further test how specific attack types respond to different model architectures when evaluated unseen.

Comparison of XGBoost on a random and an unseen-split displayed improvements in classification of web-attacks when unseen, perhaps due to similarity with the DoS attacks in the training set. A research direction could be to test different relationships between attacks: seeing how well models can classify an unseen attack Y when trained on an attack X.

I could attempt to improve my results by drawing from the latest research in the ML-IDS field. Notably, the use of online / dynamic learning models which continuously update parameters when exposed to new data. Perhaps this could be an effective tool against zero-day attacks.

LOF and iForest anomaly detectors showed that no single inductive bias dominates across attack families. iForest wins on globally-sparse attacks, LOF on locally-dense ones. By definition we can not know in advance which family the next zero-day resembles - thus a deployable IDS may attempt to leverage both biases at once through a cascade. Future work could entail creating an ensemble of detectors with heterogeneous biases (iForest, LOF, and more such as autoencoders), aggregating their scores through methods such as majority voting, averages, weighted voting etc.

A Networking Glossary

A.1 Core Concepts

Packet The basic unit of data sent over a network, containing a header (addresses, ports, flags) and a payload.

Flow A sequence of packets sharing the same **5-tuple** (source IP, destination IP, source port, destination port, protocol). Each row in CIC-IDS2017 is one "conversation", rather than raw packet data. Raw network traffic must be processed into flow data to be used on models trained on the dataset.

Fwd and Bwd Forwards and Backwards. Fwd = packets from initiator to responder. Bwd = Responses. Many features are computed separately per direction.

Port 16-bit number identifying a specific service on a host (80 is HTTP, 443 is HTTPS).

Protocol Rules governing a conversation. Relevant ones include:

- **TCP** - Transmission Control Protocol. Creates a link between devices with a three-way handshake (SYN, SYN-ACK, ACK). Handles ordering, error-correction and retransmission. Carries most application traffic.
- **UDP** - User Datagram Protocol. Fast and connectionless, sends unordered packets without asking if they arrived. Useful for DNS and streaming.
- **FTP** - File Transfer Protocol. Port 21. Old plaintext file-transfer service with password login.
- **SSH** - Secure Shell. Port 22. Encrypted remote login and command execution.
- **HTTP/HTTPS** - Hypertext Transfer Protocol (Secure). Port 80/443. Web traffic, unencrypted and encrypted respectively.

TCP flags Single bits in TCP header signaling connection state. SYN (open), ACK (acknowledge), FIN (close politely), RST (abort), PSH (deliver now), URG (urgent).

IAT Inter-Arrival Time. The gaps between consecutive packets in a flow.

A.2 CIC-IDS2017 features

Raw packet captures were converted using the **CICFlowMeter** tool into labelled CSVs of ~ 80 features. See table 10.

Group	Examples	Overview
Identifiers	Flow ID, Source/Destination IP and Port, Protocol, Timestamp	Who talked to whom, when, over what service
Duration and Counts	Flow Duration, Total Fwd/Bwd Packets, Total Length of Fwd/Bwd Packets	Conversation size and length
Packet length	Fwd/Bwd Packet Length Max/Min/Mean/Std, Packet Length Variance, Average Packet Size	Shape of the payload (or lack thereof, see PortScan)
Timing	Flow IAT Mean/Std/Max/Min, Fwd/Bwd IAT Total and stats, Active/Idle Mean/Std/Max/Min	Rhythm of the flow
Rates	Flow Bytes/s, Flow Packets/s, Fwd/Bwd Packets/s	How much data is transferred per second
Flags	FIN/SYN/RST/PSH/ACK /URG/CWE/ECE Flag Counts, Fwd/Bwd PSH and URG Flags	Connection behaviour and state
Headers and TCP internals	Fwd/Bwd Header Length, Init_Win_bytes_fwd/bwd, min_seg_size_forward, act_data_pkt_fwd	Low-level network stack specifics. Differs between operating systems
Bulk and subflow	Fwd/Bwd Avg Bytes-Bulk, Packets-Bulk, Bulk Rate, Subflow Fwd/Bwd Packets and Bytes	Sustained bulk-transfer behaviour
Ratio	Down/Up Ratio	Asymmetry in downloads and uploads
Target	Label	BENIGN or attack class

Table 10: Feature groups in CIC-IDS2017 flow representation.

A.3 Notes on Attack Classes

Brute Force - FTP/SSH-Patator Automated password guessing against FTP and SSH logins. Appears as many short, near-identical flows to port 21 or 22 from one source.

DoS (Hulk, GoldenEye, Slowhttptest, slowloris) Single-source denial of service against a web server. Hulk and GoldenEye flood with high-volume requests. Slowhttptest and slowloris do the opposite, holding connections open with deliberately slow partial requests to exhaust the connection pool.

DDoS DoS from many hosts at once, generating a large volume of concurrent flows from distinct sources.

PortScan Probing a range of ports to map open services, typically via Nmap. Produces huge numbers of tiny, short-lived flows, often with SYN sent and RST returned.

Web Attack - Brute Force / XSS / SQL Injection Attacks on a web form: credential guessing, injection scripts, injecting SQL to manipulate databases. Notably, the malicious aspects live within the payload, with minimal hints from flow statistics.

Infiltration A user on the network opens a malicious file, giving an attacker a foothold, allowing them to surveil the network through the host.

Botnet Hosts which are infected look to a command-and-control server for instructions. Typically features periodic, low-volume and regularly timed out-bound flows.

Heartbleed An OpenSSL exploit, abusing the transport layer security (TLS) "heartbeat" to read chunks of server memory.

B Machine Learning Glossary

B.1 Core Concepts

Training set Subset of original dataset used to train models.

Test set Subset of original dataset used to evaluate models

Feature Properties of each example, e.g. Source IP address, flow duration, class label.

Leakage Training a model with information it should not have e.g. training with examples found in the test set. This causes high evaluation metrics, with no indication of real-world performance.

Cross-validation set Hyper-parameter and other model tuning conducted on test set metrics causes leakage. A cross-validation (or, just validation) set is taken as a small subset of training for the purpose of leakage-free tuning.

Supervised learning Training on *labeled* data. Models learn patterns of features in order to map inputs to known outputs.

Unsupervised learning Training on *unlabeled* data. Models find hidden patterns and groupings on their own.

Anomaly detectors Models which define "normal" patterns of data and flag deviations. Can be supervised or unsupervised.

B.2 Evaluation Metrics

Accuracy Proportion of examples correctly classified.

Recall Proportion of a class' examples which are correctly classified. Can be define on a per-class basis, such as benign-recall, attack-recall etc.

Precision Proportion of examples classified as a certain class which are actually of that class. Defined on a per-class basis.

F1 Harmonic mean of recall and precision: stays low if one is low.

Macro- Unweighted average. Here, macro-attack recall is the mean of each attack class' recall.

Weighted- Weighted average. Here, weighted-attack recall is the mean of each attack class' recall scaled by the number of examples of that class present.

C CIC-IDS2017 Construction-Issues

Engelen et al. (2021)[3] expose some severe, systemic engineering flaws in the creation of the CIC-IDS2017 dataset. Some of these flaws include issues with TCP connections terminating early, splitting single conversations into multiple flows and creating distinct tool artifacts. They also discovered many labeling errors. They produced a modified version of the dataset, reconstructing or re-labeling more than 20% of the original flows. They also identified the shortcut learning features previously removed. In totality, they suggest that many of the extremely high metrics ML models achieve on the dataset are due to construction issues and software bugs. With this report's headline statistics being model failure, not success, there is no issue in the use of the dataset for the purposes of investigation, granted, with their feature selection and removal in place. I also wanted failure modes to be present on a dataset the field actually reports on.

References

- [1] Lekhika Chettri et al. “Analyzing shallow and deep classifiers in detecting unseen attacks on internet of things network”. In: *Peer Peer Netw. Appl.* 19 (2026), p. 64. URL: <https://api.semanticscholar.org/CorpusID:286659474>.
- [2] Steve Cruz et al. “Open set intrusion recognition for fine-grained attack categorization”. In: *2017 IEEE International Symposium on Technologies for Homeland Security (HST)*. 2017, pp. 1–6. DOI: 10.1109/THS.2017.7943467.
- [3] Gints Engelen, Vera Rimmer, and Wouter Joosen. “Troubleshooting an Intrusion Detection Dataset: the CICIDS2017 Case Study”. In: *2021 IEEE Security and Privacy Workshops (SPW)*. IEEE. 2021, pp. 7–12. DOI: 10.1109/SPW53761.2021.00009.
- [4] Mirzaei A. Hozouri A. and Effatparvar M. “A comprehensive survey on intrusion detection systems with advances in machine learning, deep learning and emerging cybersecurity challenges.” In: *Discov Artif Intell* 5, 314 (2025) (2025). DOI: 10.1007/s44163-025-00578-1.
- [5] Mandiant. *M-Trends 2026: Data, Insights, and Strategies From the Frontlines*. Google Cloud, Mar. 2026. URL: <https://cloud.google.com/security/resources/m-trends>.
- [6] Zach Moczko and Hany Ragab. *Do Transformers Actually Help Intrusion Detection? A Temporal Sequence Evaluation on CIC-IDS2017*. 2026. arXiv: 2606.11098 [cs.CR]. URL: <https://arxiv.org/abs/2606.11098>.
- [7] Dharmaraj Rajaram Patil et al. “Optimizing Network Intrusion Detection Systems Through Ensemble Learning and Feature Selection Using the CIC-IDS2017 Dataset”. In: *Informatika* 49.4 (Dec. 2025). DOI: 10.31449/inf.v49i4.7678. URL: <https://www.informatika.si/index.php/informatika/article/view/7678>.
- [8] Iman Sharafaldin, Arash Habibi Lashkari, and Ali A. Ghorbani. “Toward Generating a New Intrusion Detection Dataset and Intrusion Traffic Characterization”. In: *Proceedings of the 4th International Conference on Information Systems Security and Privacy - Volume 1: ICISSP*. SciTePress, 2018, pp. 108–116. DOI: 10.5220/0006639801080116.